

Lab 1:
Session A

```
(2) 1 BEGIN;  
inform 2  
reSQL 3 UPDATE film SET rental_rate = 4.99  
ggers 4 WHERE film_id = 1;  
ns (2)  
l  
ssp  
Data W  
es (1)  
l  
ons  
s (1)  
gregate  
lations  
mains  
S Confi  
S Dictio  
S Parse  
S Temp  
eign Ta  
ctions
```

Data Output Messages Notifications

WARNING: there is already a transaction in progress
UPDATE 1

Query returned successfully in 54 msec.

```
6 SELECT film_id, title, rental_rate  
7 FROM film  
8 WHERE film_id = 1;
```

Data Output Messages Notifications

Showing rows: 1 to 1 of 1

	film_id [PK] integer	title character varying (255)	rental_rate numeric (4,2)
1	1	Academy Dinosaur	4.99

Session B

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 5.99
5 WHERE film_id = 1;
```

Session B is blocked because Session A holds an exclusive lock on the same row. It must wait until Session A commits and releases the lock before its update can proceed.

```
1 BEGIN;
2
3 UPDATE film SET rental_rate = 4.99
4 WHERE film_id = 1;
5
6 SELECT film_id, title, rental_rate
7 FROM film
8 WHERE film_id = 1;
9
10 COMMIT;
```

COMMIT

Query returned successfully in 61 msec.

```
1 BEGIN;
2
3 UPDATE film
4 SET rental_rate = 5.99
5 WHERE film_id = 1;
```

UPDATE 1

Query returned successfully in 2 min 5 secs.

Session B Succeeds

```
6
7 COMMIT;
8
9 SELECT film_id, title, rental_rate
10 FROM film
11 WHERE film_id = 1;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	film_id [PK] integer	title character varying (255)	rental_rate numeric (4,2)
1	1	Academy Dinosaur	5.99

Lab 2:

Session A

Query Query History

```
1 UPDATE film
2 SET rental_rate = 4.99
3 WHERE film_id = 10;
4
5 BEGIN;
6
7 SELECT film_id, title, rental_rate
8 FROM film
9 WHERE film_id = 10;
```

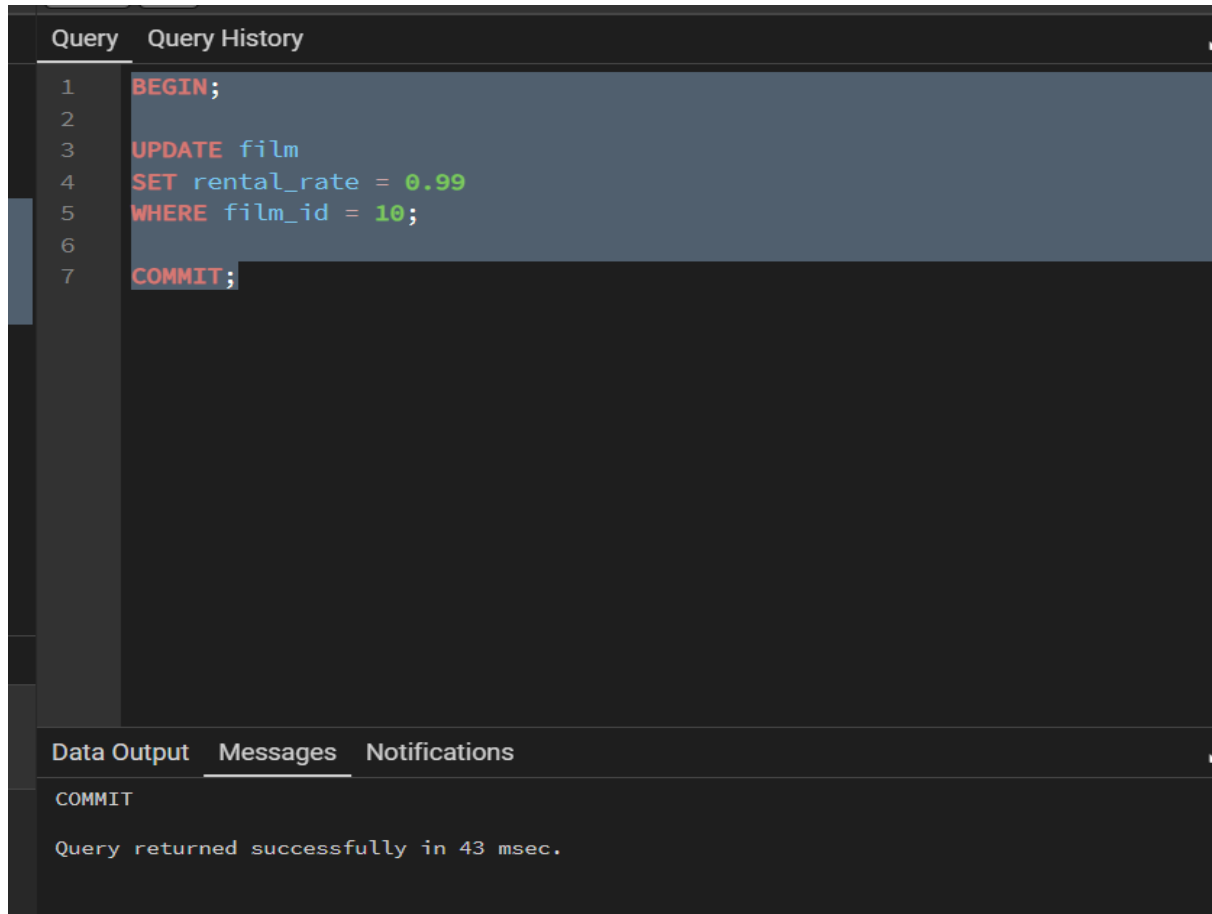
Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	film_id [PK] integer	title character varying (255)	rental_rate numeric (4,2)
1	10	Aladdin Calendar	4.99

Session A reads 4.99 initially, but after Session B updates and commits the row, the second read returns 0.99. This demonstrates a non-repeatable read under READ COMMITTED isolation.

Session B

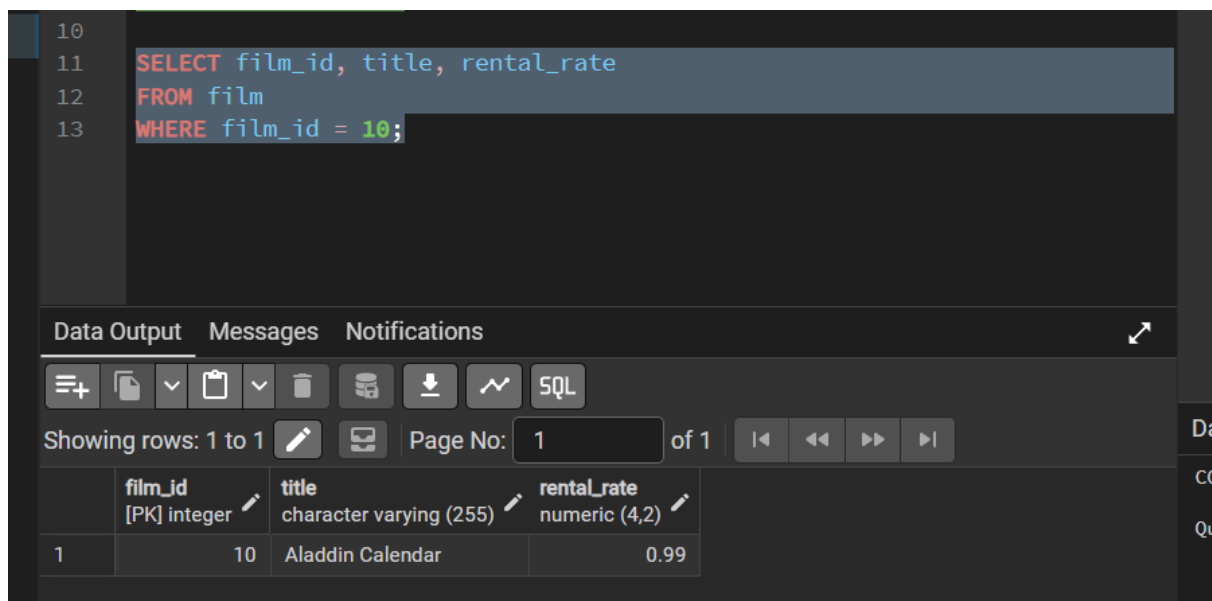


The screenshot shows a SQL IDE interface with a dark theme. The 'Query' tab is active, displaying a transaction block with the following SQL statements:

```
1 BEGIN;  
2  
3 UPDATE film  
4 SET rental_rate = 0.99  
5 WHERE film_id = 10;  
6  
7 COMMIT;
```

Below the query editor, the 'Messages' tab is selected, showing the output: 'COMMIT' and 'Query returned successfully in 43 msec.'

Return Back to Session A to read it again



The screenshot shows the same SQL IDE interface. The 'Query' tab displays a SELECT statement:

```
10  
11 SELECT film_id, title, rental_rate  
12 FROM film  
13 WHERE film_id = 10;
```

The 'Data Output' tab is active, showing the results of the query. The results are displayed in a table with the following columns: film_id, title, and rental_rate. The table contains one row of data.

	film_id [PK] integer	title character varying (255)	rental_rate numeric (4,2)
1	10	Aladdin Calendar	0.99

Lab 3

Session A & Session B: Update 1

The image shows two side-by-side screenshots of a PostgreSQL query editor. Both windows show a query with the following SQL:

```
BEGIN;  
UPDATE film  
SET rental_rate = 2.99  
WHERE film_id = 100;
```

The left window shows the query executed successfully in 52 msec. The right window shows the query executed successfully in 58 msec.

The image shows two side-by-side screenshots of a PostgreSQL query editor. The left window shows a query executed successfully in 21 secs 488 msec. The right window shows an error message:

```
ERROR: deadlock detected  
Process 18944 waits for ShareLock on transaction 1802; blocked by process 27224.  
Process 27224 waits for ShareLock on transaction 1803; blocked by process 18944.  
  
SQL state: 40P01  
Detail: Process 18944 waits for ShareLock on transaction 1802; blocked by process 27224.  
Process 27224 waits for ShareLock on transaction 1803; blocked by process 18944.  
Hint: See server log for query details.  
Context: while locking tuple (5,12) in relation "film"
```

A circular wait occurs because each session holds a row lock required by the other. PostgreSQL detects the deadlock and aborts one transaction to resolve it.

Lab 4:

Session A

The image shows a screenshot of a PostgreSQL query editor. The query is:

```
BEGIN;  
SELECT *  
FROM inventory  
WHERE inventory_id = 10  
FOR UPDATE;
```

The results are displayed in a table with the following columns: **inventory_id** [PK] integer, **film_id** smallint, **store_id** smallint, **last_update** timestamp without time zone, and **is_rented** boolean.

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone	is_rented boolean
1	10	2	2	2006-02-15 10:09:17	false

Session B is blocked

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for file operations, a dropdown menu, a filter icon, a dropdown menu, a 'No limit' dropdown, and execution buttons. Below the toolbar, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, showing a SQL script in a text editor. The script is as follows:

```
1 BEGIN;  
2  
3 SELECT *  
4 FROM inventory  
5 WHERE inventory_id = 10  
6 FOR UPDATE;
```

Below the text editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Messages' tab is active, showing a message that says 'Session B is blocked'. A tooltip 'Execute script' with the keyboard shortcut 'F5' is visible over the execution button in the toolbar.

The screenshot shows a SQL IDE interface. At the top, there is a toolbar with icons for file operations, a dropdown menu, a filter icon, a dropdown menu, a 'No limit' dropdown, and execution buttons. Below the toolbar, there are tabs for 'Query' and 'Query History'. The 'Query' tab is active, showing a SQL script in a text editor. The script is as follows:

```
8 INSERT INTO rental  
9     (rental_date, inventory_id, customer_id, staff_id)  
10 VALUES  
11     (NOW(), 10, 2, 1);  
12  
13
```

Below the text editor, there are tabs for 'Data Output', 'Messages', and 'Notifications'. The 'Data Output' tab is active, showing the result of the query:

```
INSERT 0 1
```

Below the 'Data Output' tab, there is a message that says 'Query returned successfully in 114 msec.'.

The screenshot displays a database IDE with two panels. The left panel shows a SQL query being executed in Session A:

```
1 BEGIN;  
2  
3 SELECT *  
4 FROM inventory  
5 WHERE inventory_id = 10  
6 FOR UPDATE;  
7  
8 INSERT INTO rental  
9 (rental_date, inventory_id, customer_id, staff_id)  
10 VALUES  
11 (NOW(), 10, 2, 1);  
12 COMMIT;  
13
```

The right panel shows the same SQL query in Session B, which is waiting for Session A to complete. Below the queries, the Data Output panel shows the result of Session A's query:

Showing rows: 1 to 1 of 1

	inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2	2026-09-25 15:22:58.5723

The Data Output panel also shows the message: "Query returned successfully in 94 msec."

Session B waits because Session A locks the same row using FOR UPDATE. After Session A commits, the lock is released and Session B can continue, preventing double booking.